

DEVELOPMENT ROADMAP

MIANX.AI

CORE PLATFORM

32-week phase-by-phase implementation plan for the multi-tenant, multi-domain, AI-native business operating system. Covers database, identity, authorization, AI engine, automation, billing, frontend, and Poultry OS.

Version 1.0 August 2026

BUILD THE CORE ONCE. BUILD UNLIMITED BUSINESS OS PRODUCTS ON TOP.

Table of Contents

1. Executive Summary	4
2. Phase Overview	4
3. Phase 0: Project Foundation	6
3.1 Objective	6
3.2 Tasks	6
3.3 Deliverables	6
3.4 Dependencies	6
4. Phase 1: Database and Tenancy Foundation	7
4.1 Objective	7
4.2 Database Tables	7
4.3 RLS Policies	7
4.4 Tasks	8
4.5 Definition of Done	8
5. Phase 2: Identity and Authorization	9
5.1 Objective	9
5.2 Authorization Architecture	9
5.3 Tasks	9
5.4 Security Tests Required	10
6. Phase 3: Domain and Module Engine	11
6.1 Objective	11
6.2 Database Tables	11
6.3 Domain Manifest Structure	11
6.4 Tasks	11
7. Phase 4: AI Core Foundation	13
7.1 Objective	13
7.2 AI Architecture Components	13
7.3 Database Tables	13

7.4 Tasks	14
7.5 Tool Risk Levels	14
8. Phase 5: Event and Automation Engine	15
8.1 Objective	15
8.2 Database Tables	15
8.3 Tasks	15
9. Phase 6: API and Integration Platform	17
9.1 Objective	17
9.2 API Types	17
9.3 Tasks	17
9.4 Database Tables	18
10. Phase 7: Billing and Entitlements	19
10.1 Objective	19
10.2 Subscription Lifecycle	19
10.3 Tasks	19
11. Phase 8: Frontend Platform	21
11.1 Objective	21
11.2 Core Navigation Structure	21
11.3 Tasks	21
12. Phase 9: Observability and Operations	23
12.1 Objective	23
12.2 Observability Categories	23
12.3 Tasks	23
13. Phase 10: Poultry OS Domain	25
13.1 Objective	25
13.2 Poultry Modules	25
13.3 Database Tables	25
13.4 Poultry AI Agents	26
13.5 Tasks	26

14. Phase 11: Security Hardening and Production Readiness	27
14.1 Objective	27
14.2 Security Testing Categories	27
14.3 Tasks	27
15. Phase Dependency Graph	29
Appendix A: Core v1 Definition of Done	30

1. Executive Summary

Mianx.ai is a multi-tenant, multi-domain, AI-native Business Operating System designed to serve as a universal platform foundation. Rather than building separate software products for each industry vertical, Mianx.ai provides a shared Core platform with pluggable Domain packages. The first production domain is Poultry OS, followed by Restaurant, Retail, and Manufacturing in future phases. This roadmap defines the complete implementation plan across 12 sequential phases spanning 32 weeks, covering every layer from database architecture and tenant isolation to AI agents, billing, automation, and the production frontend.

The architecture follows a strict principle: **Build the Core once. Build unlimited Business OS products on top.** This means the Core must be domain-agnostic, fully tenant-isolated, and extensible enough that adding a new business domain requires zero changes to the Core itself. If the architecture supports Poultry cleanly without any Poultry-specific code inside Core, then the design is validated and ready for additional domains.

Each phase in this roadmap includes specific deliverables, database tables to be created, API endpoints to be implemented, integration points, and a definition-of-done checklist. Phases are designed to be sequential because each builds on the foundation established by the previous phase. However, within each phase, tasks are organized for maximum parallelism where dependencies allow. The technology baseline uses Next.js with React and TypeScript on the frontend, Next.js server-side architecture for the backend, PostgreSQL via Supabase for the database, Supabase Auth for authentication, and a provider-agnostic AI abstraction layer for all artificial intelligence capabilities.

2. Phase Overview

The following table provides a high-level summary of all 12 phases, their duration, primary focus area, and the key deliverable that marks the completion of each phase. This serves as a quick reference for project planning and milestone tracking. Each phase is described in detail in its own section later in this document, including specific tasks, database tables, dependencies, and success criteria.

Phase	Name	Duration	Primary Focus	Key Deliverable
0	Project Foundation	Week 1-2	Setup and scaffolding	Runnable dev environment
1	Database and Tenancy	Week 2-4	PostgreSQL schema and RLS	Tenant-isolated database
2	Identity and Authorization	Week 4-6	Auth, RBAC, permissions	Secure access control system
3	Domain and Module Engine	Week 6-8	Domain registry, manifests	Extensible domain framework
4	AI Core Foundation	Week 8-12	AI router, agents, tools	Governed AI runtime

Phase	Name	Duration	Primary Focus	Key Deliverable
5	Event and Automation	Week 12-14	Events, workflows, jobs	Reliable automation engine
6	API and Integration	Week 14-16	APIs, webhooks, OAuth	External connectivity layer
7	Billing and Entitlements	Week 16-18	Plans, subscriptions, usage	Commercial SaaS foundation
8	Frontend Platform	Week 18-22	App shell, design system	Complete UI platform
9	Observability and Ops	Week 22-24	Logging, monitoring, alerts	Operational visibility
10	Poultry OS Domain	Week 24-30	Industry modules, agents	First production domain
11	Production Readiness	Week 30-32	Security, testing, deploy	Production-ready system

Table 1: Phase overview with durations and key deliverables

3. Phase 0: Project Foundation

Duration: Week 1-2

PHASE 0

PROJECT FOUNDATION

Week 1-2

3.1 Objective

The Foundation phase establishes the development environment, repository structure, and all tooling required for subsequent phases. This includes scaffolding the Next.js application with TypeScript, configuring the Supabase connection, setting up Prisma as the ORM, establishing environment variable management, and creating the basic folder structure that mirrors the conceptual architecture defined in the specification documents. No business logic is implemented in this phase; the sole focus is on creating a clean, well-organized, and runnable development environment that every subsequent phase can build upon reliably.

3.2 Tasks

- Initialize Next.js 16 project with TypeScript, Tailwind CSS, and App Router configuration
- Configure Supabase client (both browser and server-side) with environment variables
- Set up Prisma ORM with PostgreSQL connection and migration framework
- Establish repository folder structure: apps/web, core/, ai/, automation/, domains/, database/
- Configure ESLint, Prettier, Husky pre-commit hooks, and TypeScript strict mode
- Set up environment variable management with .env.local and validation schema
- Create basic health-check API route and verify full dev-to-deploy pipeline
- Set up Vercel deployment configuration and verify preview/production deployments
- Install core dependencies: shadcn/ui, Vercel AI SDK, and utility libraries
- Document development workflow, branching strategy, and PR templates

3.3 Deliverables

Deliverable	Acceptance Criteria
Next.js project scaffold	Runnable app with /health endpoint returning 200
Repository structure	All directories created per architecture spec section 25
CI/CD pipeline	Vercel preview deployments on PR, production on merge to main
Development documentation	README with setup instructions and architecture overview

Table 2: Phase 0 deliverables

3.4 Dependencies

No prior phases are required. This phase depends only on having a Supabase project provisioned and a Vercel account configured. The developer must have Node.js 20+, PostgreSQL access credentials, and appropriate Supabase and Vercel authentication tokens available in the local environment before starting this phase.

4. Phase 1: Database and Tenancy Foundation

Duration: Week 2-4

PHASE 1 DATABASE AND TENANCY

Week 2-4

4.1 Objective

This phase establishes the PostgreSQL database schema as the authoritative system of record for all Mianx.ai data. Every tenant-owned record must carry an explicit `organization_id`, and Row Level Security (RLS) policies must enforce tenant isolation at the database level. The database architecture follows the conceptual separation defined in the Data Platform specification: Core tables (organizations, users, memberships, roles, permissions) live alongside Domain tables in a single PostgreSQL cluster with logical separation. This phase also implements the Prisma data-access layer, the UUID-based ID strategy, and the audit log table that will capture all meaningful mutations throughout the platform lifecycle.

4.2 Database Tables

Table	Key Columns
organizations	id (UUID), name, slug, status, timezone, locale, currency, created_at, updated_at
profiles	id, display_name, avatar_url, locale, timezone, created_at, updated_at
organization_memberships	id, organization_id, user_id, status, joined_at, UNIQUE(org, user)
teams	id, organization_id, name, description, created_at, updated_at
team_members	team_id, membership_id, UNIQUE(team, membership)
roles	id, organization_id (nullable), name, slug, description, is_system
permissions	id, key (e.g. organization.view), description, created_at
role_permissions	role_id, permission_id, UNIQUE(role, permission)
membership_roles	membership_id, role_id, UNIQUE(membership, role)
settings	id, organization_id, scope_type, scope_id, key, value
files	id, organization_id, uploaded_by, name, storage_path, mime_type, size
audit_logs	id, organization_id, actor_type, actor_id, action, resource_type, resource_id, metadata
notifications	id, organization_id, recipient_user_id, type, title, body, data, read_at

Table 3: Core database tables created in Phase 1

4.3 RLS Policies

Every tenant-owned table receives Row Level Security policies. The core pattern uses a PostgreSQL helper function `user_has_org_access(user_id, organization_id)` that checks for an active membership record. SELECT policies use the USING clause, INSERT policies use WITH CHECK to validate the new row `organization_id` belongs to the authorized organization, and UPDATE policies use both USING and WITH CHECK to prevent cross-tenant data movement. DELETE policies verify organization ownership before allowing removal. These policies are defense in depth and complement the application-level authorization implemented in Phase 2, not replace it.

4.4 Tasks

- Create initial Prisma schema with all Core tables and UUID primary keys
- Implement `organization_id` foreign key relationships and unique constraints
- Write and test RLS policies for all tenant-owned tables (SELECT, INSERT, UPDATE, DELETE)
- Create `user_has_org_access()` PostgreSQL helper function
- Implement Prisma client extensions for tenant-scoped queries
- Set up database migration workflow: design, migrate, validate, review, production
- Create seed script for development data (test organizations, users, roles)
- Write cross-tenant isolation tests: 5 mandatory test categories per spec
- Implement `audit_logs` trigger function for automatic mutation tracking
- Verify all indexes on `organization_id`, composite keys, and frequently queried columns

4.5 Definition of Done

Phase 1 is complete when all Core tables exist in the PostgreSQL database with proper constraints and indexes, every tenant-owned table has working RLS policies verified by automated tests, the Prisma client can perform CRUD operations within a tenant scope, cross-tenant access tests pass with zero failures, audit logging captures all mutations, and the migration pipeline works reliably from development through to production deployment.

5. Phase 2: Identity and Authorization

Duration: Week 4-6

PHASE 2 IDENTITY AND AUTHORIZATION

Week 4-6

5.1 Objective

This phase implements the complete identity and authorization system that every subsequent phase depends on. Authentication is handled by Supabase Auth (email/password initially, with extensibility for OAuth and SSO), while authorization is a layered system built on top of the database tables created in Phase 1. The authorization context resolution follows a strict chain: Authentication, then Organization Membership, then Role, then Permission, then Resource Ownership, then Action. The permission format follows the pattern domain.resource.action (e.g., poultry.flock.view). AI agents use the same authorization system and are never granted administrator access by default. The fail-closed principle is enforced throughout: any missing authorization information results in an automatic denial.

5.2 Authorization Architecture

Layer	Description
Authentication	Supabase Auth verifies user identity (JWT token validation)
Organization Membership	Active membership record links user to organization
Role Assignment	Membership has one or more roles (system or custom)
Permission Check	Role maps to permissions (domain.resource.action format)
Resource Ownership	User must own or be authorized for the specific resource
Action Authorization	The requested action must be in the allowed permission set

Table 4: Authorization layer chain

5.3 Tasks

- Implement Supabase Auth integration (sign-up, sign-in, sign-out, session management)
- Build organization creation flow with slug generation and initial admin role assignment
- Implement invitation system: email invite, accept/reject, membership activation
- Create RBAC middleware for Next.js API routes (resolve user, org, roles, permissions)
- Build permission-check utility: has_permission(user, org, permission_key)
- Implement organization context middleware (reject client-supplied organization_id)
- Create role management UI (system roles: Owner, Admin, Member, Viewer)
- Build team management: create teams, add/remove members, team-scoped permissions
- Implement session management with revocation support and secure cookie handling

- Write authorization test suite: RBAC, ABAC, cross-tenant denial, role isolation
- Create service-role boundary enforcement: no service key in browser or AI context
- Implement storage authorization with path pattern: organizations/{org_id}/files/{file_id}

5.4 Security Tests Required

Test	Description
Cross-tenant SELECT	User from Org A cannot read Org B data via API or direct DB
Cross-tenant UPDATE	Cannot change organization_id to move records between tenants
Permission denial	User without poultry.flock.view gets 403 on flock endpoints
AI agent denial	AI agent without finance permissions cannot access financial data
Suspended membership	Suspended member cannot access any organization resources
Removed membership	Immediately after removal, all access is revoked
Session revocation	Revoked sessions cannot make authenticated requests
Role escalation	Member role cannot promote itself to Owner or Admin

Table 5: Mandatory security tests for Phase 2

6. Phase 3: Domain and Module Engine

Duration: Week 6-8

PHASE 3 DOMAIN AND MODULE ENGINE

Week 6-8

6.1 Objective

The Domain Engine is what makes Mianx.ai a multi-domain platform rather than a single-application product. A domain is a self-contained business capability package that registers itself with the Core through a manifest file. The manifest declares the domain name, version, modules, permissions, workflows, agents, dashboards, and settings. This phase implements the domain registry, the manifest validation system, the module lifecycle (draft, available, enabled, disabled, deprecated), and the organization-level activation/deactivation flow. The critical architectural constraint is that the Core must never contain domain-specific business rules. If the Poultry domain is activated and functions correctly without any poultry-specific code existing inside Core, the architecture is working as designed.

6.2 Database Tables

Table	Key Columns
domains	id, name, slug, version, description, status, manifest (JSONB), created_at
organization_domains	id, organization_id, domain_id, status, configuration (JSONB), activated_at, UNIQUE(org, domain)
modules	id, domain_id, name, slug, version, description, manifest (JSONB), status
organization_modules	id, organization_id, module_id, status, configuration (JSONB), activated_at, UNIQUE(org, module)

Table 6: Domain and Module engine database tables

6.3 Domain Manifest Structure

Each domain ships with a manifest JSON that describes its complete capability set. The manifest includes the domain name, slug, version, description, an array of module definitions (each with its own name, slug, permissions, and settings schema), workflow definitions, agent definitions, dashboard configurations, and a list of Core dependencies. When an organization activates a domain, the system validates the manifest, checks Core version compatibility, verifies dependencies, creates the organization_domain record, initializes default configuration, registers all modules, runs any required setup migrations, and finally activates the domain. Deactivation follows the reverse process but never automatically destroys business data; data remains recoverable in the archived state.

6.4 Tasks

- Create domain and module database tables with RLS policies
- Build domain registry service: register, update, deprecate, archive domains
- Implement manifest validation engine (schema validation, dependency checking)
- Create organization domain activation flow with entitlement checking
- Build module lifecycle management: enable, disable, configure per organization
- Implement domain-scoped settings system (organization-level overrides)
- Create permission registration API (domains declare permissions on activation)
- Build navigation generation from active domain modules
- Write domain isolation tests (cross-domain access denied by default)
- Create a skeleton "Poultry" domain as architectural validation

7. Phase 4: AI Core Foundation

Duration: Week 8-12

PHASE 4 AI CORE FOUNDATION

Week 8-12

7.1 Objective

The AI Core is what transforms Mianx.ai from a traditional SaaS platform into an AI-native Business Operating System. This phase implements the provider-agnostic AI abstraction layer, the model router, the agent registry, the tool system with typed input/output contracts, the agent permission system, execution limits, AI usage tracking, and the foundations of memory and knowledge retrieval. AI agents are first-class security principals with scoped capabilities, not unrestricted chatbots. Every AI action is governed by the same authorization system used for human users, and every AI run produces an auditable usage record. The AI architecture supports multiple agent types: internal, customer-facing, domain-specific, workflow-integrated, analytics, and autonomous background agents.

7.2 AI Architecture Components

Component	Description
Provider Abstraction	Unified interface wrapping OpenAI, Anthropic, Google, and local models
Model Router	Selects model based on task type, quality, latency, cost, and policy
Agent Registry	Stores agent definitions: identity, instructions, tools, permissions, limits
Tool System	Typed tools with server-side execution and authorization checks
Memory Engine	Tenant-scoped memory: user, organization, agent, conversation levels
Knowledge Retrieval	Authorized RAG pipeline with organization-scoped vector search
Usage Tracking	Per-organization, per-agent, per-model token and cost recording
Governance Layer	Policy checks, risk classification, approval requirements, audit trail

Table 7: AI Core architecture components

7.3 Database Tables

Table	Key Columns
agents	id, organization_id, domain_id, name, slug, description, status, configuration (JSONB)
agent_tools	id, agent_id, tool_key, configuration (JSONB), enabled
agent_permissions	agent_id, permission_key (scoped to domain.resource.action)
ai_usage_records	id, organization_id, agent_id, model, provider, input_tokens, output_tokens, cost, duration_ms
agent_memories	organization_id, agent_id, scope, content, metadata
knowledge_sources	id, organization_id, domain_id, type, status, configuration

Table 8: AI Core database tables

7.4 Tasks

- Implement provider abstraction layer with OpenAI and Anthropic adapters
- Build model router with task-type routing, fallback chains, and cost optimization
- Create agent registry CRUD service with organization and domain scoping
- Implement tool registry with typed input/output schemas and risk level classification
- Build tool authorization middleware (every tool call checks agent permissions)
- Create agent runtime with execution limits (max_steps, max_token_budget, max_cost)
- Implement AI usage tracking with per-request cost estimation and recording
- Build agent permission system reusing Core RBAC (agents as security principals)
- Create memory engine with tenant-scoped storage (never cross organization boundaries)
- Implement knowledge retrieval pipeline with authorized vector search
- Build AI request context normalization (request_id, user, org, domain, permissions)
- Create AI error classification system (MODEL_UNAVAILABLE, TOOL_DENIED, BUDGET_EXCEEDED)
- Implement fail-safe chain: retry, fallback model, safe deterministic path, human escalation
- Write AI isolation tests: agent cannot access unauthorized domain data

7.5 Tool Risk Levels

Risk Level	Description and Examples
LOW	Read-only operations, no side effects (get_flock_metrics, view_report)
MEDIUM	Send notifications, create drafts (send_notification, create_draft)
HIGH	Create business records, financial operations (create_sale, record_payment)
CRITICAL	Change ownership, delete data, modify permissions (change_org_owner)

Table 9: AI tool risk classification levels

8. Phase 5: Event and Automation Engine

Duration: Week 12-14

PHASE 5 EVENT AND AUTOMATION

Week 12-14

8.1 Objective

This phase implements the deterministic automation backbone of Mianx.ai. Business events are immutable facts emitted by core actions (payment.created, inventory.low, workflow.completed). These events flow through a transactional outbox pattern to guarantee reliable delivery, and can trigger workflows, AI actions, notifications, analytics updates, and external integrations. The workflow engine supports complex multi-step processes with conditions, human approval gates, AI decision steps, delays, retries, and compensating actions. Every workflow execution is tenant-isolated and can never dynamically switch organization context mid-execution. The system supports bounded retries with exponential backoff and jitter, dead-letter queues for exhausted retries, and idempotency keys to prevent duplicate side effects.

8.2 Database Tables

Table	Key Columns
events	id, event_type, event_version, organization_id, domain_id, source_type, source_id, correlation_id, payload, occurred_at
outbox	id, organization_id, event_type, payload, status, attempts, available_at, published_at
workflow_definitions	id, organization_id, domain_id, name, slug, status, definition (JSONB), trigger
workflow_runs	id, workflow_id, organization_id, status, input, output, current_step, started_at, completed_at, error
workflow_step_runs	id, workflow_run_id, step_id, status, attempt, input, output, error
workflow_approvals	id, workflow_run_id, organization_id, requested_action, requested_by, decision, reason, expires_at
jobs	id, organization_id, type, payload, status, priority, scheduled_at, attempts

Table 10: Event and Automation engine database tables

8.3 Tasks

- Implement event emission system with stable envelope (type, version, org, correlation_id)
- Build transactional outbox pattern (business change + outbox event in same DB transaction)
- Create event publisher that reads outbox and publishes to event bus reliably
- Implement workflow definition engine with JSON-based step and condition definitions

- Build workflow execution engine with state machine (queued, running, waiting, completed, failed)
- Create human approval gate system with expiration and delegation support
- Implement AI decision steps with structured output validation before workflow use
- Build retry policy engine with bounded attempts, exponential backoff, and jitter
- Create dead-letter queue handling with operator review and manual retry interface
- Implement job scheduling system with cron-based and event-triggered execution
- Write workflow isolation tests (no cross-tenant execution, no context switching)
- Implement idempotency for all side-effecting workflow actions

9. Phase 6: API and Integration Platform

Duration: Week 14-16

PHASE 6 API AND INTEGRATION

Week 14-16

9.1 Objective

This phase builds the secure external connectivity layer that allows Mianx.ai to communicate with the outside world. The API platform implements typed RESTful endpoints with a standard response envelope, cursor-based pagination, idempotency keys for side-effecting operations, and comprehensive error contracts. The integration framework supports OAuth 2.0 connections for external services, incoming and outgoing webhooks with HMAC signature verification, API key management for programmatic access, and an adapter pattern that isolates external provider logic from Core business logic. Every API endpoint enforces the full authorization chain established in Phase 2, and the tenant context is always resolved from trusted server-side data, never from client-supplied request bodies. External failures are handled through circuit breakers and must never corrupt internal business state.

9.2 API Types

API Type	Description
Public API	External-facing, versioned (/api/v1/), rate-limited, idempotent
Internal API	Server-to-server within Mianx.ai, no rate limiting
Domain API	Domain-specific endpoints, dynamically registered per active domain
Admin API	Platform administration, requires platform-level RBAC
Webhook Endpoint	Inbound webhooks with signature verification and replay protection
AI Tool API	Internal API used by AI agents, same authorization as user APIs

Table 11: API type classifications

9.3 Tasks

- Implement standard API response envelope: {data, meta, request_id}
- Build cursor-based pagination for all list endpoints (next_cursor, has_more)
- Create API key management with scopes, expiration, and organization scoping
- Implement idempotency middleware with Idempotency-Key header support
- Build webhook delivery system with HMAC signing and retry with backoff
- Create inbound webhook endpoint with signature verification and event deduplication
- Implement OAuth 2.0 connection flow for external service integration
- Build integration adapter pattern with health monitoring and status tracking
- Create circuit breaker implementation (Closed, Open, Half-Open state machine)

- Implement request/correlation ID propagation across APIs, workflows, and AI runs
- Build data export pipeline with authorization, async processing, and download expiry
- Write API contract tests and integration security tests (SSRF, replay, permission boundary)

9.4 Database Tables

Table	Key Columns
api_keys	id, organization_id, name, scopes (JSONB), status, expires_at, last_used_at
integration_connections	id, organization_id, provider, status, scopes, credential_reference, health_status
webhook_deliveries	id, organization_id, event_type, endpoint, status, attempts, response, delivered_at
idempotency_records	key, organization_id, endpoint, request_hash, status, response, expires_at

Table 12: API and Integration database tables

10. Phase 7: Billing and Entitlements

Duration: Week 16-18

PHASE 7 BILLING AND ENTITLEMENTS

Week 16-18

10.1 Objective

This phase implements the commercial SaaS foundation that enables Mianx.ai to operate as a subscription-based platform. The billing architecture cleanly separates three concerns: Entitlements (what a customer is commercially allowed to use), Authorization (what a user is security-allowed to do), and Policy (operational constraints). Plans define commercial packages with versioned feature sets. Subscriptions link organizations to plans with lifecycle management (trialing, active, past_due, grace_period, paused, cancelled). Usage metering tracks consumption against limits with idempotent recording to prevent double-charging. AI usage receives special attention with dedicated meters for tokens, tool calls, and agent runs, enabling AI budget controls (monthly limits, per-agent limits, warn-then-restrict behavior). Plan downgrades never silently delete customer data; the system either blocks the downgrade, schedules it with a cleanup window, or allows it while restricting new creation.

10.2 Subscription Lifecycle

State	Description
trialing	Free trial period with full or limited features, time-limited
active	Paid subscription, all entitled features accessible
past_due	Payment failed, grace period begins
grace_period	7-day window to update payment, full access maintained
paused	Organization paused access, data preserved
cancelled	User cancelled, access until end of billing period
expired	Billing period ended after cancellation, access restricted
suspended	Platform-initiated suspension for ToS violations

Table 13: Subscription lifecycle states

10.3 Tasks

- Create plans and plan_versions tables with version locking for commercial integrity
- Implement subscription management with full lifecycle state machine
- Build entitlement engine: plan features mapped to organization capabilities
- Create usage metering service with idempotent recording (idempotency_key per event)
- Implement AI usage meters: requests, input/output tokens, tool calls, agent runs
- Build AI cost tracking with per-organization, per-agent, per-model visibility
- Create AI budget controls with warn-then-restrict behavior and upgrade prompts
- Implement plan upgrade/downgrade flows with data preservation guarantees

- Build feature flags system (separate from entitlements and authorization)
- Create payment provider abstraction adapter for Stripe integration
- Implement billing reconciliation (periodic comparison of Mianx state vs provider state)
- Build billing failure handling with controlled grace and suspension behavior

11. Phase 8: Frontend Platform

Duration: Week 18-22

PHASE 8

FRONTEND PLATFORM

Week 18-22

11.1 Objective

This phase builds the complete frontend platform that users interact with. The frontend follows a layered component architecture: Foundation (design tokens, base styles), Primitives (Button, Input, Select), Composites (Form, DataTable, Dialog), Patterns (PermissionGuard, OrgSwitcher, DomainNav), Domain Components (Poultry-specific), and Pages. The app shell provides persistent navigation with an organization switcher, domain switcher, search, AI assistant access, notifications, and user menu. Navigation is dynamically generated from the active organization domains and enabled modules, filtered by the current user permissions. The AI Workspace provides a conversation panel with context display, attachment support, tool visibility, and clear distinction between AI recommendations and executed actions. Code splitting ensures domain bundles load only when needed: Core Shell loads first, then the selected domain bundle, then individual module bundles on navigation. The design system supports RTL languages (Urdu, Arabic) through logical layout properties, and meets WCAG accessibility standards for keyboard navigation and screen reader support.

11.2 Core Navigation Structure

Navigation	Description
Home	Dashboard with organization summary, active domains, recent activity
My Business	Organization management, team, branding, settings
Domains	Active domain cards linking to domain dashboards
AI	AI Workspace with conversation panel and agent management
Automations	Workflow list, run history, job scheduling
Analytics	Organization-level analytics and reporting
Integrations	Connection management, webhook configuration
Team	Members, roles, invitations, teams
Billing	Subscription, usage, invoices, plan management
Settings	Organization settings, notifications, security

Table 14: Core navigation structure

11.3 Tasks

- Build design token system: colors, typography, spacing, radius, shadows, breakpoints
- Implement core component library with 28+ components (Button, Input, Table, Dialog, etc.)
- Create app shell with header, sidebar navigation, and main workspace area

- Build organization switcher with membership verification and context refresh
- Implement permission-aware navigation (hide/disable items based on user permissions)
- Create domain switcher showing only entitled and authorized domains
- Build AI Workspace with conversation panel, context display, and action confirmation
- Implement dashboard framework with configurable KPI cards, charts, and tables
- Create domain code splitting (Core Shell, domain bundles, module lazy loading)
- Build state architecture (server state, UI state, form state, URL state, AI state)
- Implement loading, empty, error, and permission-denied states for every screen
- Create responsive layout system (layout changes, not just shrinking)
- Build RTL readiness using logical layout properties (margin-inline, padding-inline)
- Implement accessibility: keyboard navigation, semantic HTML, screen reader support
- Create organization onboarding flow (create org, configure, invite team, select domain)

12. Phase 9: Observability and Operations

Duration: Week 22-24

PHASE 9 OBSERVABILITY AND OPERATIONS

Week 22-24

12.1 Objective

This phase implements the operational visibility layer that enables the Mianx.ai team to monitor, diagnose, and respond to production issues. Observability follows the three pillars model (logs, metrics, traces) supplemented by additional signals including errors, events, AI telemetry, and cost telemetry. Structured logging in JSON format captures timestamp, level, service, event, organization_id, request_id, trace_id, and error_code for every meaningful operation. The correlation ID system propagates request_id, trace_id, organization_id, and user_id across APIs, workflows, AI runs, integration calls, and audit logs, enabling end-to-end request tracing. The Command Center provides a platform administration dashboard for managing organizations, users, domains, subscriptions, AI usage, AI cost, security events, and system health. AI-specific observability tracks quality signals (tool success rate, task completion, human approval rate) and safety signals (policy denials, prompt injection detections, agent loop detection, excessive retries).

12.2 Observability Categories

Category	Key Signals
Platform Health	CPU, memory, connections, queue depth, DB latency, cache hit rate
Application Health	Request count, latency distributions, error rates by endpoint
Business Health	Active organizations, users, flocks, workflows, revenue metrics
AI/Cost Health	Cost per model, org, domain, agent; AI gross margin calculation
AI Quality	Tool success rate, task completion, human approval/rejection rate, hallucination rate
AI Safety	Policy denials, tool auth failures, prompt injection detections, agent loops

Table 15: Observability signal categories

12.3 Tasks

- Implement structured JSON logging with correlation ID propagation
- Create error tracking integration with stack traces and context capture
- Build application metrics: request latency histograms, error rates, throughput
- Implement AI usage dashboard with per-organization cost visibility
- Build AI quality monitoring (tool success, task completion, approval rates)
- Create AI safety monitoring (policy denials, injection detection, loop detection)
- Implement health check endpoints for all critical services
- Build Command Center dashboard with organization, domain, and AI management views

- Create alert routing system with priority levels (P1-P4) and escalation policies
- Implement incident model (Detected, Acknowledged, Investigating, Mitigating, Resolved)
- Build telemetry data redaction (no passwords, API keys, tokens, or personal data in logs)
- Create SLO tracking framework with error budget calculations

13. Phase 10: Poultry OS Domain

Duration: Week 24-30

PHASE
10

POULTRY OS DOMAIN

Week 24-30

13.1 Objective

Poultry OS is the first production domain and the ultimate architectural validation of the Mianx.ai Core. If the Core supports Poultry cleanly, without any Poultry-specific hacks inside Core code, then the multi-domain architecture is proven and ready for Restaurant, Retail, and Manufacturing domains. This phase implements the complete Poultry business package including eight core modules: Farm management, Shed management, Flock management, Feed tracking, Health records, Production metrics, Procurement, and Sales. Each module follows the domain manifest pattern, registers its own permissions, and integrates with the AI, automation, and billing systems already built in the Core. Poultry-specific AI agents are created with scoped tools and permissions, and Poultry-specific workflows automate common business processes like feed scheduling, health alerts, and sales recording.

13.2 Poultry Modules

Module	Description
Farm	Manage farm locations, capacity, contact info, and operational status
Shed	Track shed types, capacity, environmental conditions, and occupancy
Flock	Manage flock lifecycle: placement, growth, mortality, weight tracking
Feed	Record feed consumption, conversion ratios, stock levels, and costs
Health	Track vaccinations, treatments, mortality causes, and health alerts
Production	Monitor egg production, body weight, feed conversion, and growth curves
Procurement	Manage chick procurement, feed purchases, medicine, and equipment
Sales	Record sales transactions, customer management, and revenue tracking

Table 16: Poultry OS modules

13.3 Database Tables

Table	Key Columns
poultry_farms	id, organization_id, name, location, capacity, status, contact_info
poultry_sheds	id, organization_id, farm_id, name, shed_type, capacity, current_count
poultry_flocks	id, organization_id, shed_id, breed, placement_date, quantity, status
poultry_feed_records	id, organization_id, flock_id, date, feed_type, quantity, cost
poultry_health_records	id, organization_id, flock_id, date, type, treatment, veterinarian
poultry_mortality_records	id, organization_id, flock_id, date, count, cause, notes

Table	Key Columns
poultry_production_records	id, organization_id, flock_id, date, eggs_collected, weight
poultry_sales	id, organization_id, customer_id, date, items (JSONB), total_amount, currency

Table 17: Poultry OS database tables

13.4 Poultry AI Agents

Agent	Tools	Permissions
Flock Manager	get_flock_metrics, get_mortality, create_alert	poultry.flock.view, poultry.alert.create
Feed Optimizer	get_feed_usage, get_production, recommend_feed	poultry.feed.view, poultry.report.generate
Health Monitor	get_health_records, get_mortality_trends, escalate	poultry.health.view, poultry.alert.create
Sales Analyst	get_sales_data, get_revenue, forecast_demand	poultry.sale.view, finance.report.view

Table 18: Poultry OS AI agents

13.5 Tasks

- Create Poultry domain manifest with all 8 modules, permissions, and settings
- Implement Farm and Shed management with CRUD APIs and RLS policies
- Build Flock lifecycle management (placement, growth tracking, depletion)
- Create Feed tracking with consumption recording and conversion ratio calculations
- Implement Health records with vaccination schedules and mortality tracking
- Build Production metrics dashboard with egg collection and growth curves
- Create Procurement module for chick, feed, and medicine purchasing
- Implement Sales module with customer management and revenue tracking
- Build Poultry-specific AI agents with scoped tools and permissions
- Create Poultry workflows (feed scheduling, health alerts, sales recording)
- Build Poultry dashboards with KPI cards, charts, and AI insights
- Verify zero Poultry-specific code exists inside Core (architecture validation test)
- Write end-to-end Poultry business flow tests

14. Phase 11: Security Hardening and Production Readiness

Duration: Week 30-32

**PHASE
11**

PRODUCTION READINESS

Week 30-32

14.1 Objective

The final phase transforms the development system into a production-ready platform. This encompasses comprehensive security testing across twelve categories (authentication, RBAC, ABAC, tenant isolation, RLS, API, session management, secret exposure, AI tool authorization, prompt injection, integration security, and audit completeness), performance optimization, production infrastructure configuration, backup and disaster recovery verification, and operational readiness. Every security test must pass with zero failures before the system can be deployed to production. The Command Center must be fully operational for platform administration. Documentation must be complete enough that a new team member can understand the architecture, set up the development environment, and make changes without verbal knowledge transfer. The definition of done from the Architecture Specification is the ultimate checklist: if every item passes, Mianx.ai Core v1 is ready.

14.2 Security Testing Categories

Category	Test Scope
Authentication	Session fixation, token theft, brute force, social engineering vectors
RBAC	Role escalation, permission bypass, privilege inheritance validation
ABAC/Policy	Context-based access, conditional denial, approval gate bypass
Tenant Isolation	Cross-tenant SELECT, UPDATE, DELETE, and organization reassignment
RLS	Direct database access bypass attempts, policy gap analysis
API	Injection, mass assignment, IDOR, rate limit bypass, parameter tampering
Session	Session hijacking, fixation, revocation timing, cookie security
Secret Exposure	Service key in browser, secrets in AI context, debug data leakage
AI Tool Auth	Agent exceeding permission scope, tool parameter injection
Prompt Injection	External content treated as untrusted, instruction override attempts
Integration	SSRF, payload validation, credential exposure, replay attacks
Audit	Mutation without audit record, audit record tampering, completeness check

Table 19: Production security testing categories

14.3 Tasks

- Run all 12 security testing categories with documented pass/fail results

- Perform threat modeling for AI agents, payments, integrations, file uploads, admin
- Implement MFA support for admin and platform-level operations
- Conduct performance testing and optimization (API latency, DB queries, page load)
- Set up production infrastructure with separate database credentials per service role
- Implement backup and recovery verification (database, object storage, AI data)
- Configure production monitoring dashboards and alert routing
- Create incident response runbooks for P1-P4 alert categories
- Complete operational documentation (architecture, setup, deployment, runbooks)
- Verify all audit trails are complete, tamper-resistant, and queryable
- Conduct load testing to verify system behavior under expected peak traffic
- Perform tenant onboarding/offboarding verification (data export, deletion, cleanup)
- Final architecture review: verify no domain-specific code exists in Core
- Obtain sign-off on Definition of Done checklist (all 17 items from Architecture Spec)

15. Phase Dependency Graph

The following table maps the explicit dependencies between phases. A phase can only begin when all its dependencies have reached their definition of done. Some phases have partial dependencies where specific deliverables from an earlier phase are needed, but the full phase does not need to be complete. These partial dependencies are noted in the description column. Understanding these dependencies is critical for project scheduling because they define the critical path through the 32-week timeline. Any delay on the critical path phases (0, 1, 2, 4, 8, 10) directly impacts the final delivery date.

Phase	Depends On	Notes
Phase 0	None	No dependencies, can start immediately
Phase 1	Phase 0	Requires runnable project and Supabase connection
Phase 2	Phase 1	Requires database tables and RLS policies
Phase 3	Phase 2	Requires authorization middleware and permissions
Phase 4	Phase 2	Requires authorization for agent permissions (partial Phase 2)
Phase 5	Phase 1, Phase 2	Requires database events and authorization (partial Phase 2)
Phase 6	Phase 2, Phase 5	Requires auth middleware and event system
Phase 7	Phase 3	Requires domain/module system for feature entitlements
Phase 8	Phase 2, Phase 3	Requires auth, domains, and modules for UI rendering
Phase 9	Phase 4, Phase 5	Requires AI telemetry and event/audit data
Phase 10	Phase 3, Phase 4, Phase 5	Requires domain engine, AI, and automation
Phase 11	All phases	Complete system must be built and tested

Table 20: Phase dependency graph

Appendix A: Core v1 Definition of Done

The following checklist is taken directly from Section 28 of the Mianx.ai Architecture Specification. Every item must be verified as complete before the Core platform is considered ready for Phase 1 domain implementation. This checklist serves as the ultimate quality gate between Core development and production domain deployment. Each item should have a corresponding test or verification procedure that can be executed automatically or manually with documented evidence.

Checklist Item	Verification
User can register and login	Authentication flow works end-to-end with Supabase Auth
Organization can be created	Organization creation with slug, settings, and admin assignment
Users can join organizations	Invitation, acceptance, and membership activation flow
Roles work correctly	System and custom roles with permission mapping
Permissions work correctly	domain.resource.action format enforced on all endpoints
Tenant isolation works	Zero cross-tenant data access verified by automated tests
Domains can be registered	Domain manifest validation and registration in Core
Domains can be activated	Organization-level activation with entitlement checking
Modules can be activated	Module-level enable/disable with configuration
Events work reliably	Transactional outbox, event bus, and consumer processing
Audit works completely	All mutations captured with actor, resource, before/after
AI agents have controlled permissions	Agent authorization scoped and enforced
AI usage is tracked	Per-organization, per-agent, per-model usage recording
Basic workflows execute	Multi-step workflows with conditions and actions complete
Billing/entitlements have foundation	Plans, subscriptions, and feature gating operational
Command Center can manage platform	Admin dashboard for organizations, domains, subscriptions
Security tests pass	All 12 security testing categories with zero failures
Core can accept a new domain	New domain registers without Core code changes

Table 21: Core v1 Definition of Done checklist